

Object Oriented Programming with Java 1

Exam Project

Sheep Escape

Sune Groennebaek

SETUR

May 2026

1 Introduction

Sheep Escape is a terminal-based game written in Java. The player controls a sheep escaping a farm through 4 levels, avoiding enemies, collecting items and disarming traps. It is rendered with ANSI escape codes and emoji sprites.

This report describes the OOP concepts used, how they appear in the code, and the testing strategy. Each section includes a code example.

2 Application Description

The game has 4 levels. Level 1 is loaded from a text file; levels 2-4 are randomly generated mazes. The player moves with w/a/s/d + Enter, collects items (hay, bombs, treats) and avoids two enemy types: Farmer (chases the player) and Sheepdog (patrols a fixed route). The goal is to reach the gate on each level and ultimately the Freedom tile on the last level.

The codebase has 20 Java classes organized in three packages: core (Position, Direction, Color, Canvas, Entity, Player, Dungeon), entity (15 Entity subclasses), and app (Game, DungeonLoader, MazeGenerator).

3 Implementation

3.1 Inheritance

Inheritance lets a class reuse fields and methods from a parent class. In this project, the abstract class Entity defines position, sprite, colors, solidity, and default methods (renderOn, onStep, onTurn). All 15 game objects inherit from Entity and only override what they need.

For example, ElectricFence extends Entity and overrides onStep() to deal damage. Everything else -- rendering, position, solidity -- is inherited from Entity:

```
public class ElectricFence extends Entity {
    private int interactions = 3;

    public ElectricFence(Position position) {
        super(position, '~', Color.ORANGE, Color.DARK_GREEN,
false);
        this.sprite = "electric-emoji";
    }
}
```

```

    }

    @Override
    public void onStep(Player player, Dungeon dungeon) {
        player.damage(10);
        interactions--;
        if (interactions <= 0) {
            dungeon.removeEntity(this);
        }
    }
}

```

The `super()` call passes values to `Entity`'s constructor. `ElectricFence` only adds what is unique to it: a hit counter and damage logic. This pattern is used by all 15 subclasses -- `StoneWall`, `Farmer`, `HayItem`, `Gate`, `Player`, etc.

3.2 Polymorphism

Polymorphism means calling a method on a variable of type `Entity`, and Java automatically runs the correct subclass version. The `Dungeon` class uses this everywhere -- it stores all game objects in a single `List<Entity>` and calls `onStep()` or `onTurn()` without knowing the concrete type.

The key example is `Dungeon.move()`. When the player steps onto a new tile, the method calls `e.onStep()` on every entity at that position:

```

// From Dungeon.move()
player.moveTo(newPos);

List<Entity> atPosition = getEntitiesAt(newPos);
for (Entity e : atPosition) {
    if (e != player) {
        e.onStep(player, this);
    }
}

```

If the entity is an `ElectricFence`, Java runs `ElectricFence.onStep()` which deals damage. If it is a `HayItem`, it runs `HayItem.onStep()` which adds the item to inventory. If it is a `Gate`, it triggers a level transition. The `Dungeon` code does not contain a single `if/else` or `instanceof` check -- dynamic dispatch handles it all.

3.3 Encapsulation

Encapsulation means hiding internal state behind methods. Fields are private and can only be changed through controlled methods that enforce

rules.

The Player class is a good example. Health is private, and the damage() and heal() methods ensure it never goes below 0 or above maxHealth:

```
public class Player extends Entity {
    private int health = 100;
    private int maxHealth = 100;
    private List<Entity> inventory = new ArrayList<>();

    public void damage(int amount) {
        health = Math.max(health - amount, 0);
    }

    public void heal(int amount) {
        health = Math.min(health + amount, maxHealth);
    }

    public boolean isDead() { return health <= 0; }
}
```

External code cannot set health to -50 or 999 -- it must go through damage() and heal(). The same principle applies to Dungeon, where the entity list is private and only accessible through methods like getEntityAt() and isSolidAt().

3.4 Collections

Java Collections (ArrayList, List) are used in two central places: Dungeon stores all game objects in a List<Entity>, and Player stores picked-up items in a List<Entity> inventory.

The inventory system shows polymorphism and collections working together. useItem() removes an item from the list and returns it, so Game can call the type-specific use() method:

```
public Entity useItem(int index) {
    if (index >= 0 && index < inventory.size()) {
        return inventory.remove(index);
    }
    return null;
}
```

Because inventory is a List<Entity>, it can hold HayItem, BombItem and TreatItem in the same list. Game then uses instanceof to determine which use() method to call.

3.5 Separation of Concerns

The project separates UI from game logic. Game handles all I/O: reading input with Scanner, building the frame with StringBuilder, and writing to the terminal. Dungeon handles all mechanics: movement, collision, entity interactions, and enemy AI. Dungeon has no imports of Scanner, System.out, or any I/O class.

This means the input and rendering system could be replaced -- for example with a GUI -- without changing any game logic. The same separation exists between Entity subclasses: each entity defines its own behavior (onStep, onTurn), so adding a new entity type does not require changes to Dungeon or Game.

3.6 Emoji Sprite System

The assignment uses single characters for entities: '@' for the player, '#' for walls, etc. This project replaces them with emoji sprites -- the player is a sheep emoji, the farmer is a person emoji, walls are block characters, and so on. This gives the game a more visual and distinctive look compared to plain ASCII.

The challenge is that emojis occupy 2 terminal columns while a normal character occupies 1. If some cells are 1 column and others are 2, the grid becomes misaligned. The solution is to make every cell exactly 2 columns wide. Canvas stores a String per cell (not a char), and each entity sets a 2-column sprite in its constructor:

```
// Entity constructor stores a 2-column default
this.sprite = symbol + " ";

// Subclasses override with emojis (also 2 columns)
this.sprite = "sheep-emoji"; // Player
this.sprite = "farmer-emoji"; // Farmer
this.sprite = "dog-emoji"; // Sheepdog
this.sprite = "flag-emoji"; // Gate
```

Canvas.render() then outputs each cell's sprite directly, producing a grid where every column is exactly 2 terminal characters wide. This also means the board is visually twice as wide as the number of logical columns -- a 17-column level occupies 34 terminal columns -- which gives more room for the gameplay to breathe.

Background color is set per cell using ANSI 24-bit RGB escape codes. This matters even for emoji cells, because emojis do not fully cover the pixel area

-- the background color shows through at the edges. Floor entities use only a green background with two spaces as their sprite, creating a solid-colored tile.

3.7 Flicker-Free Rendering

The assignment suggests clearing the screen and printing each cell individually. This causes visible flicker because the terminal redraws after every print call. Fixing this required several iterations.

The first fix was batching: `Canvas.render()` builds the entire board as one `String`, and `Game` assembles everything (clear-screen, board, stats, messages) into a single `StringBuilder`. This fixed levels 1-3 but not level 4.

The problem was `PrintStream`'s internal 8 KB buffer. Level 4's frame with ANSI color codes is roughly 19,000 characters, so `PrintStream` flushes mid-frame. The fix was bypassing `PrintStream` entirely:

```
private OutputStream rawOut =
    new FileOutputStream(FileDescriptor.out);

private void printFrame(StringBuilder frame) {
    rawOut.write(frame.toString().getBytes("UTF-8"));
    rawOut.flush();
}
```

This writes the entire frame as raw bytes in a single OS-level `write()` call. The terminal receives the complete frame atomically -- no intermediate states, no flicker.

4 Testing

The project has 59 JUnit 5 tests across 7 test classes. All tests are black-box: they call public methods and verify observable results.

- `PlayerTest` (12): health, inventory, movement, edge cases
- `DungeonTest` (14): movement, collision, entity lookup, win/lose
- `ElectricFenceTest` (6): damage, deactivation after 3 hits
- `SnareTest` (5): hidden state, one-time activation
- `FarmerTest` (6): chase range, damage, treat distraction
- `SheepdogTest` (6): route following, cyclic patrol, damage
- `DungeonLoaderTest` (10): file parsing, entity placement

As an example, this test verifies that `ElectricFence` deactivates after exactly

3 interactions:

```
@Test
void testDeactivatesAfterThreeHits() {
    fence.onStep(player, dungeon);
    fence.onStep(player, dungeon);
    fence.onStep(player, dungeon);
    assertEquals(0, fence.getInteractions());
    assertFalse(dungeon.getEntities().contains(fence));
}
```

5 Conclusion

The Entity hierarchy demonstrates inheritance and polymorphism in a practical setting -- `Dungeon.move()` and `Dungeon.tick()` operate on entities uniformly without type checks. Encapsulation keeps state consistent (`Player.damage()` cannot produce negative health). The separation between Game and Dungeon keeps I/O and logic independent. The flicker-free rendering required going beyond the standard library (bypassing `PrintStream`) but resulted in smooth output on all levels.

6 Shortcomings

- Farmer AI is a greedy one-step chase with no pathfinding
- Bombs only destroy Fence entities, not other types
- No save/load functionality
- AlarmSensor always faces SOUTH